Journal of Signal Processing Systems (2025) 97:185–195
https://doi.org/10.1007/s11265-025-01966-7

Real-time and Resource-efficient Embedded Computer Vision Via Optimizing Lightweight CNNs for FPGA Acceleration

Ahmad Mouri Zadeh Khaki[1] · **Ahyoung Choi**[1]

Received: 23 April 2025 / Revised: 13 June 2025 / Accepted: 23 July 2025 / Published online: 28 July 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract
Deploying deep learning models on resource-constrained edge devices requires efficient hardware acceleration to balance accuracy, latency, and resource utilization. This study optimizes and implements three lightweight convolutional neural networks —MobileNetV2, SqueezeNet, and ResNet18—on field-programmable gate array (FPGA) using transfer learning with the PyTorch framework of Xilinx Vitis AI. Through hardware-aware quantization, model pruning, and FPGA-specific optimizations, we achieve significant reductions in inference latency and resource consumption while maintaining high classification accuracy. To comprehensively evaluate performance, we compare our FPGA implementation against CPU and GPU alternatives. Experimental results show that our optimized models achieve Top-1 accuracies of 91.05%, 89.27%, and 94.73% for MobileNetV2, SqueezeNet, and ResNet18, respectively, on the CIFAR-10 dataset, with inference latencies of 10 ms, 6.28 ms, and 19.95 ms/frame. Detailed FPGA implementation and resource utilization analysis demonstrate the scalability and efficiency of our approach. These findings highlight the potential of FPGA-based deep learning deployment for real-time AI applications in autonomous systems and edge computing.

Keywords FPGA acceleration · Edge AI deployment · Embedded computer vision · Hardware-aware optimization · Lightweight CNNs · Resource-constrained inference

1 Introduction

Deep learning (DL) has revolutionized artificial intelligence (AI) applications, particularly in computer vision for real-time decision-making tasks. Convolutional neural networks (CNNs) have demonstrated remarkable performance in these tasks, but their deployment on resource-constrained edge devices remains a significant challenge due to high computational and memory demands. Traditional implementations on CPUs and GPUs often consume substantial power and require dedicated hardware, limiting their feasibility for real-time, low-power applications such as autonomous systems, Internet of Things (IoT) devices, and mobile computing [1–3].

Field-programmable gate arrays (FPGAs) have emerged as a promising alternative for signal and image processing [4–7] and edge AI acceleration [8, 9], offering advantages such as low power consumption, reconfigurability, and parallel computing capabilities. Moreover, FPGAs enable hardware-specific optimizations, making them well-suited for real-time AI inference in embedded systems. However, deploying CNNs on FPGA presents significant challenges, including model quantization-induced accuracy degradation, resource constraints, and high computational complexity.

Traditional FPGA-based implementations often rely on high-level synthesis (HLS) or hardware description languages (HDL) to design custom architectures. While HDL-based implementations offer fine-grained control over hardware resources, they require extensive expertise, are time-intensive, and are not easily adaptable for transfer learning. HLS-based approaches, on the other hand, abstract hardware complexities but often result in suboptimal efficiency and resource utilization, limiting their scalability for DL tasks [10–13]. These challenges underscore the need for framework-driven FPGA acceleration, where pre-optimized software tools facilitate efficient deployment of pretrained

✉ Ahmad Mouri Zadeh Khaki
ahmadmouri@gachon.ac.kr

[1] Department of AI and Software, Gachon University, Seongnam-si 13120, South Korea

CNN models through transfer learning without extensive low-level hardware modifications.

To address this, frameworks such as Xilinx Vitis AI [14] have been developed to streamline FPGA-based DL deployment. While Vitis AI provides hardware-aware optimizations such as quantization and pruning, previous studies often sacrifice accuracy for resource/power efficiency or vice versa. For instance, previous works have implemented basic CNN models via custom implementation [15] or transfer learning [16]. Also, lightweight models like ResNet and MobileNet [17] FPGA implementation have been presented, but without comprehensive performance benchmarking from multiple perspectives. This trade-off creates a gap in achieving an optimal balance between accuracy, latency, and resource utilization for real-time applications. Additionally, many existing works lack detailed implementation methodologies, limiting their scalability and reproducibility [18].

This study addresses these challenges by optimizing and implementing three lightweight CNN architectures—MobileNetV2, SqueezeNet, and ResNet18—on FPGA using transfer learning with the PyTorch framework of Xilinx Vitis AI for classifying the CIFAR-10 dataset [19]. To address limitations found in our TensorFlow-based prior work [20], our approach utilizes the PyTorch DL framework which offers enhanced compatibility with pretrained models and supports a wider range of features, including BatchNormalization layers [14]. We integrate hardware-aware quantization, model pruning, and FPGA-specific optimizations to achieve low-latency, high-accuracy inference while minimizing resource consumption. To comprehensively evaluate the performance of our approach, we compare FPGA implementations against both CPU and GPU alternatives, offering a detailed analysis of trade-offs in efficiency and scalability. The key contributions of this work include:

- A detailed workflow for integrating transfer learning with FPGA acceleration using PyTorch and Xilinx Vitis AI.
- Comprehensive performance benchmarks, demonstrating significant reductions in inference latency and resource utilization while maintaining high classification accuracy.
- An in-depth FPGA resource utilization analysis, providing insights into the scalability and efficiency of our approach for real-time AI applications.

Experimental results demonstrate that our optimized models achieve high classification accuracy while significantly reducing inference latency and resource consumption. Additionally, we provide a detailed implementation methodology, addressing the shortcomings of previous Vitis AI-based works and offering a scalable, framework-driven solution for deploying CNNs on FPGA. These findings underscore the potential of FPGA acceleration of DL models for real-time AI applications in autonomous systems, IoT, and edge computing. Our findings offer a foundation for future FPGA-based AI acceleration and can enhance real-time inference in mobile and edge applications with improved efficiency.

The rest of the paper is structured as follows. Section 2 presents the methodology, including hardware and software tools, transfer learning strategies, and FPGA-specific optimizations. Section 3 discusses experimental results, comparing FPGA performance with CPU and GPU implementations. Section 4 provides a detailed discussion of findings, including key insights and future research directions. Finally, Sect. 5 concludes the paper.

2 Proposed Design Methodology

This section details the workflow for designing and implementing MobileNetV2, SqueezeNet, and ResNet18 on FPGA for image classification. The methodology includes CNN adaptation, model training, hardware-specific optimizations, and FPGA deployment, ensuring an optimal balance between accuracy, inference latency, and resource utilization. This study employs transfer learning, quantization, and pruning using the PyTorch framework within Xilinx Vitis AI, facilitating an efficient deployment pipeline tailored for real-time edge computing applications.

Figure 1 illustrates the proposed workflow for deploying pretrained CNN models on FPGA. The process starts with data preprocessing and augmentation to improve model performance. During model training, pretrained models from PyTorch are fine-tuned on the CIFAR-10 dataset using transfer learning. These training steps are conducted within an Integrated Development Environment (IDE) such as Jupyter Notebook [21]. The trained models are then quantized to 8-bit integer precision, reducing computational complexity and enhancing hardware efficiency. Next, the models are compiled using the Vitis AI compiler, converting them into FPGA-compatible binaries. These binaries are deployed to the FPGA's Deep Learning Processing Unit (DPU), where real-time inference takes place. Finally, the models are tested on 10,000 CIFAR-10 images, and key performance metrics—including accuracy, inference latency, and resource utilization—are analyzed.

This workflow simplifies FPGA deployment for DL models, eliminating the need for low-level HDL or HLS coding while maintaining efficiency. By leveraging PyTorch in Vitis AI, this approach enhances model compatibility and ensures scalability for future AI applications on edge devices.

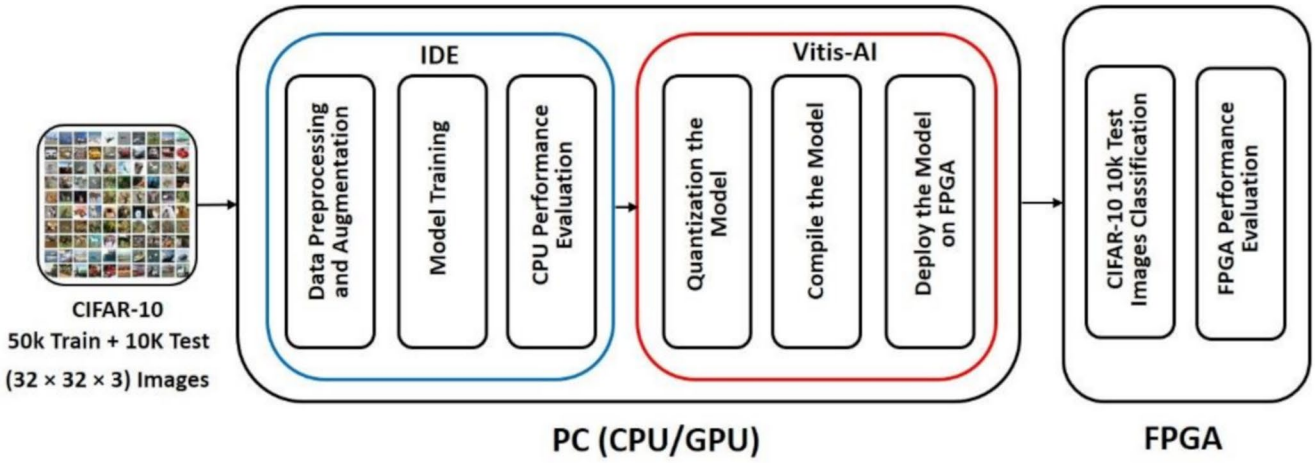


Fig. 1 The proposed workflow for deploying pretrained CNN models on FPGA

2.1 Software and Hardware Tools

The models were developed and trained on a PC running Python 3.12.9 with the PyTorch framework. The system was equipped with 16 GB of RAM and an Intel i5-13400 F 2.50 GHz processor (referred to as CPU), which was used for data preprocessing and initial model development. Additionally, a NVIDIA GeForce RTX 4060 (referred to as GPU) with CUDA 12.6 tools were utilized to accelerate model training and fine-tuning, providing significant computational efficiency for DL tasks.

For FPGA deployment, the Xilinx Vitis AI 2.5.0 toolkit was employed to optimize and compile the models. The experiments were conducted using an Avnet Ultra96-V2 evaluation board, which integrates a Zynq UltraScale+MPSoC FPGA. This FPGA features a combination of programmable logic (PL) and ARM-based processing units (PS), making it well-suited for DL inference. The board is equipped with a DPU of DPUCZDX8G with a B1600 configuration, enabling efficient acceleration of CNN inference workloads. In our implementation, the DPU is responsible for executing all computational layers of the CNN models, including convolutional, pooling, and fully connected layers. The PS is used for non-intensive tasks such as loading preprocessed input data, initiating inference, and collecting output results using the Python-based PYNQ framework. This setup ensures that core inference computation is performed entirely within the FPGA's PL, leveraging the parallelism and low-latency characteristics of the DPU.

2.2 Dataset and Preprocessing

This study utilizes the CIFAR-10 dataset, which consists of 60,000 color images (32×32 pixels) evenly distributed across 10 classes. CIFAR-10 is widely used for benchmarking image classification models, particularly in resource-constrained environments such as FPGAs [22–24]. The dataset is split into 50,000 images for training and 10,000 images for testing to evaluate model performance. To further improve model evaluation and prevent overfitting, the training set was further divided randomly into 40,000 images for training and 10,000 images for validation.

To ensure compatibility with the pretrained CNN models on ImageNet, all images were resized to 224×224 pixels, aligning with the input requirements of MobileNetV2, SqueezeNet, and ResNet18. Various data augmentation techniques including random flips and rotations were applied to improve model generalization and robustness. Also, the images were normalized to a range of (-1,1), and processed in batches of 64.

2.3 CNN Models and Training Process

This study employs three lightweight CNN architectures—MobileNetV2, SqueezeNet, and ResNet18—for image classification. We utilize the PyTorch framework, which supports BatchNormalization layers, allowing for improved convergence and stability during training.

As illustrated in Fig. 2, the pretrained models were adapted to the CIFAR-10 dataset by replacing the fully connected layers of *Classifier* with custom layers tailored for the 10-class classification task. We did not freeze any layers during training process, allowing all layers to be updated during training to fine-tune the entire network. This strategy enhances model performance by adapting both low-level and high-level features to the new dataset.

The models were trained for 10 epochs using the Stochastic Gradient Descent (SGD) optimizer with a learning rate of 0.01. To prevent overfitting, we implemented early stopping and learning rate reduction. Early stopping monitored validation loss and halted training if no improvement was observed for 5 epochs, ensuring computational efficiency.

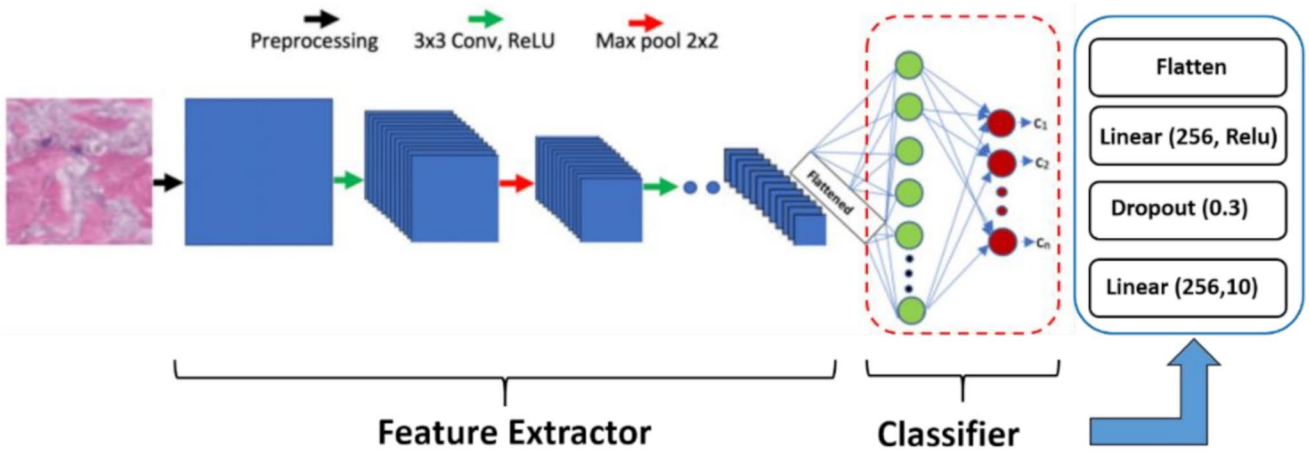


Fig. 2 CNN model architectures implemented in this work using transfer learning

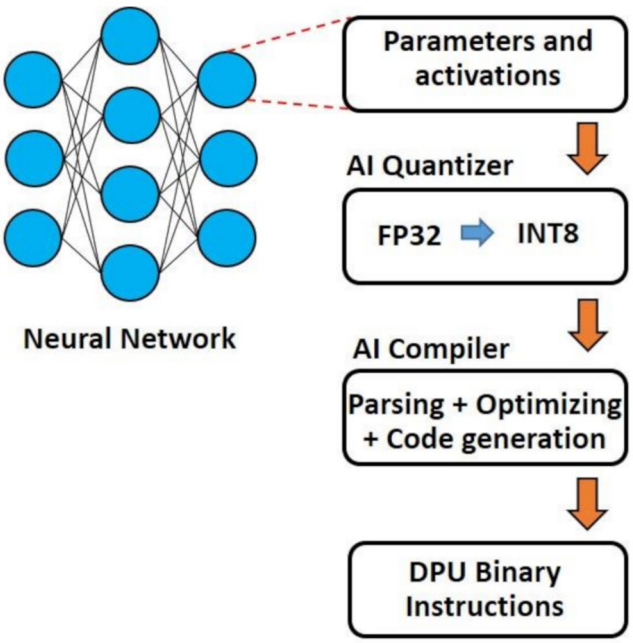


Fig. 3 The Xilinx Vitis-AI pipeline for optimizing and deploying AI models on FPGA

Additionally, learning rate reduction dynamically decreased the learning rate by a factor of 0.5 when validation performance plateaued for 3 epochs, allowing for smoother convergence and improved generalization. The loss function used for training was CrossEntropyLoss, which is well-suited for multi-class classification tasks.

This training strategy effectively balances accuracy, computational cost, and generalization, ensuring that the models are well-optimized for FPGA deployment while maintaining competitive performance.

2.4 Models Optimization and Deployment on FPGA

Xilinx Vitis AI is a comprehensive framework designed for deploying DL models on Xilinx FPGAs, enabling efficient acceleration through hardware-aware optimizations. It supports frameworks such as PyTorch and TensorFlow, streamlining the integration of pretrained models into FPGA-based systems. The conceptual pipeline of Vitis AI is depicted in Fig. 3.

In this study, after building and training the MobileNetV2, SqueezeNet, and ResNet18 models on a PC, the models were imported into the Vitis AI platform for FPGA deployment. A systematic workflow was followed to optimize hardware resource utilization while preserving model accuracy, consisting of the following steps:

1) **Quantization**: To improve efficiency, we applied post-training quantization using Xilinx Vitis AI tools, converting 32-bit floating-point weights and activations into 8-bit fixed-point precision. This significantly reduces memory footprint and computational complexity while maintaining classification accuracy. Before quantizing the models, hardware and quantization-aware optimization was performed to ensure minimal accuracy degradation during inference.
2) **Compilation**: Once quantized, the models were compiled using the Vitis AI compiler, generating FPGA-compatible binaries optimized for the Zynq UltraScale+MPSoC architecture. This step included adapting the models for execution on the DPU, tailoring operations for parallel execution and enhancing hardware-specific performance.
3) **Deployment on FPGA**: The compiled models were deployed onto the Avnet Ultra96-V2 FPGA board, where the DPU configuration was fine-tuned to leverage FPGA-specific hardware resources, including Flip-flops

(FFs), Look-Up Tables (LUTs), Digital Signal Processors (DSPs), and Block RAM (BRAM). These optimizations maximized computational efficiency while minimizing inference latency.

4) **Model Execution Using the PYNQ Framework**: To execute the models, we utilized the Python-based PYNQ framework within Jupyter Notebook, enabling seamless interaction with the FPGA. Test images from the CIFAR-10 dataset were imported, preprocessed, and transmitted to the FPGA's PL for inference. The classified outputs were retrieved from the PS, completing the real-time inference workflow.

This systematic optimization and deployment process ensures efficient DL acceleration on FPGA, enabling real-time inference in resource-constrained environments with enhanced computational performance and scalability.

2.5 Evaluation Metrics

The performance of the implemented models was assessed using accuracy, inference latency, and resource utilization as key evaluation metrics. Specifically, Top-1 and Top-5 accuracy were used to measure classification performance. Top-1 accuracy represents the percentage of instances where the model's highest probability prediction matches the true label, and Top-5 accuracy measures the percentage of instances where the true label is among the model's top 5 highest probability predictions. These metrics are formally defined as follows:

$$\text{Top - 1 Accuracy}\,(\%) = \frac{1}{n}\sum_{i=1}^{n} 1\left(\hat{y}_i = y_i\right) \times 100 \tag{1}$$

$$\text{Top - 5 Accuracy}\,(\%) = \frac{1}{n}\sum_{i=0}^{n} 1\left(\hat{y}_i \in P_i^5\right) \times 100, \tag{2}$$

where n is the total number of test samples, $\hat{y}_i$ is the predicted label, y_i is the ground truth label, and P_i^5 represents the set of the top 5 predicted labels for the i-th sample.

To provide a comprehensive baseline, the performance of FPGA-based implementations was compared against both CPU and GPU-based inference using the same MobileNetV2, SqueezeNet, and ResNet18 models. Additionally, resource utilization was analyzed, focusing on the usage of FFs, LUTs, DSPs, and BRAM to evaluate the efficiency of the FPGA in handling computational workloads.

This multi-faceted evaluation provides insights into the trade-offs between accuracy, latency, and hardware efficiency, demonstrating the potential of FPGA acceleration for real-time, resource-constrained applications, such as autonomous systems and IoT devices.

3 Experimental Results

This section presents a comprehensive evaluation of the implemented MobileNetV2, SqueezeNet, and ResNet18 models on FPGA. The evaluation focuses on classification accuracy, inference latency and FPGA resource utilization to determine the trade-offs between efficiency and performance in real-time edge applications. The CIFAR-10 test set, consisting of 10,000 images, was used to evaluate all models.

3.1 Classification Accuracy

Table 1 presents the Top-1 and Top-5 accuracy of MobileNetV2, SqueezeNet, and ResNet18 before quantization (CPU/GPU) and after quantization (FPGA). The results indicate that all three models maintain high classification performance, with only a minor decrease in accuracy due to quantization.

Despite the quantization process converting 32-bit floating-point parameters to 8-bit fixed-point precision, the Top-1 accuracy loss remained below 4%, while Top-5 accuracy decreased by less than 0.1% across all models. MobileNetV2 experienced the most significant drop in Top-1 accuracy (-3.49%), which can be attributed to its extensive use of depthwise separable convolutions, which are more sensitive to quantization effects. In contrast, ResNet18 retained 94.73% Top-1 accuracy, exhibiting the highest resilience to quantization.

To further analyze classification performance, Fig. 4 presents the confusion matrices of the FPGA-implemented models on the CIFAR-10 test set. These matrices provide a detailed breakdown of how each class is classified, highlighting potential misclassifications introduced by quantization. The results indicate that misclassifications primarily occur between visually similar categories, which suggests

Table 1 Classification accuracy before and after quantization in this work

Model	Top-1 Accuracy (%)			Top-5 Accuracy (%)		
	Pre-quant.	Post-quant.	Change (%)	Pre-quant.	Post-quant	Change (%)
MobileNetV2	94.54	91.05	94.54	99.92	99.86	-0.06
SqueezeNet	89.94	89.27	89.94	99.74	99.71	-0.03
ResNet18	94.87	94.73	94.87	99.92	99.90	-0.02

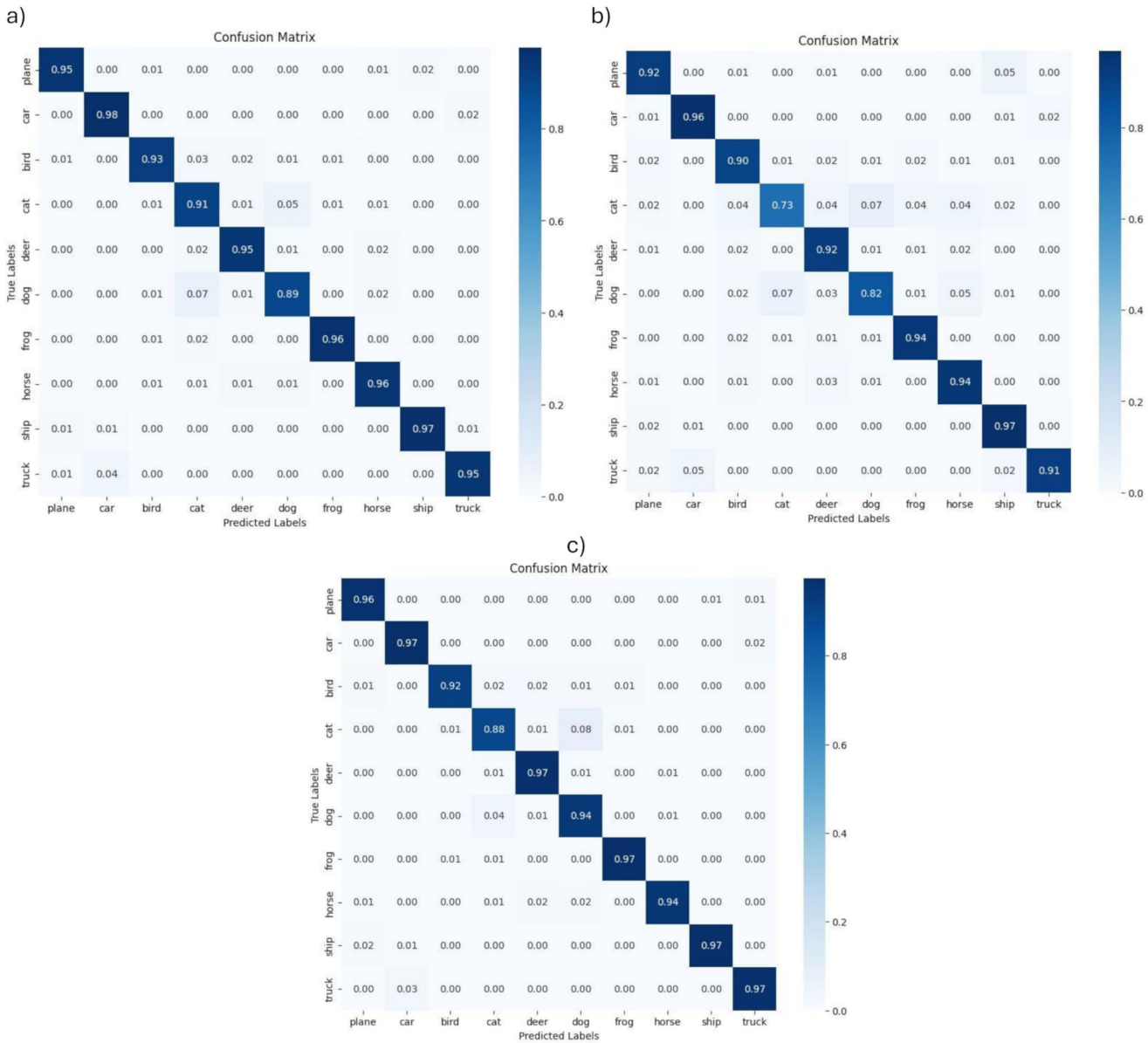


Fig. 4 Confusion matrix on CIFAR-10 test set classification using our proposed design. (**a**) MobileNetV2, (**b**) SqueezeNet, and (**c**) ResNet18

Table 2 Inference latency comparison across CPU, GPU, and FPGA

Model	CPU(s)	GPU(s)	FPGA(s)	Speed-up vs. CPU	Speed-up vs. GPU
MobileNetV2	332.57	13.64	100.4	3.31×	0.14×
SqueezeNet	166.73	10.23	62.80	2.65×	0.16×
ResNet18	403.54	12.08	199.5	2.02×	0.06×

that accuracy degradation due to quantization is minimal and does not significantly alter the model's decision boundaries.

3.2 Inference Latency

Inference latency is a critical factor in evaluating the suitability of DL models for real-time applications. Table 2 presents the total inference time required to process the CIFAR-10 test set across different hardware platforms, including CPU, GPU, and FPGA. The inference latency reported in Table 2 reflects the total time to process 10,000 CIFAR-10 test images end-to-end, including all steps from input preparation to output retrieval including inter-unit data transfer and model execution. For fairness, the same model architecture was used across CPU, GPU, and FPGA. Quantization was applied where appropriate: FPGA used INT8

precision via post-training quantization, while CPU and GPU used FP32 inference. In each case, the same functionality was executed, with no difference in model structure or layer operations. These settings ensure a fair and functionally equivalent comparison across platforms.

The results indicate that FPGA implementation significantly reduces inference time compared to CPU-based execution, achieving a speedup of up to 3.31× for MobileNetV2. However, GPUs consistently outperform FPGAs in terms of latency, with achieving up to 19× lower latency than CPU and outperforming FPGA by a factor of 1.4–3.5×, depending on the model.

Among the three models, SqueezeNet exhibited the lowest inference latency on FPGA (62.80s), making it the most efficient model for deployment, while ResNet18 required the longest execution time (199.5s) due to its deeper architecture and higher computational complexity. The performance of MobileNetV2 on FPGA (100.4s) falls between the other two models, reflecting its balanced design for accuracy and efficiency.

It should be noted that although GPU inference remains the fastest, FPGAs are known to allow for flexible, low-power deployment in embedded and edge applications due to their reconfigurable architecture and energy-efficient parallel computation capabilities [25–27], where high-performance GPUs may not be practical due to power and thermal constraints. These findings highlight the trade-offs between hardware efficiency, runtime, and deployment feasibility, positioning our design as a promising candidate for energy-efficient AI inference in autonomous systems and IoT devices. Although power consumption was not directly measured in this work, future studies will incorporate power profiling to validate the expected energy savings of FPGA-based deployment.

3.3 FPGA Resource Usage

Hardware resource utilization is a crucial metric in evaluating the efficiency of FPGA-based DL deployments, particularly in resource-constrained environments. Table 3 summarizes the FPGA resource utilization for FPGA deployment of DL models. In this work, the resource usage was identical across all models because the default DPUCZDX8G B1600 configuration was used without modification to ensure a fair and consistent comparison across all models, avoiding variations caused by different hardware settings.

Table 3 FPGA hardware resource usage in this work

Resource	Utilization	Available	Utilization %
LUT	38,418	70,560	54.5%
FF	58,831	141,120	41.7%
BRAM	126	216	58.3%
DSP	326	360	90.6%
RAM (MB)	162	2 GB	8.1%

From the results, it can be inferred that the proposed design utilizes a high portion of DSP units (90.6%) due to the computationally intensive nature of CNN operations, particularly matrix multiplications in convolutional layers. LUT usage remains moderate at 54.5%, demonstrating efficient use of FPGA logic resources without excessive overhead. BRAM utilization (58.3%) indicates a balanced use of on-chip memory, ensuring that the models effectively manage intermediate computations. FFs were used efficiently (41.7%) to distribute processing tasks across available logic elements, while RAM usage remained minimal (8.1%), highlighting the FPGA's effective memory management strategy.

The observed resource allocation suggests that the default DPU settings prioritize inference latency reduction by heavily utilizing DSPs for acceleration, while ensuring that logic resources and memory are efficiently distributed. Although the same configuration was used for all models, reconfiguring the DPU architecture could further optimize resource usage for different CNNs, depending on specific accuracy, latency, and power constraints. For example, such adjustments could allow lighter architectures like SqueezeNet to operate with reduced DSP utilization, or allocate additional resources to deeper networks like ResNet18 to enhance inference throughput.

These results confirm that our approach for FPGA deployment successfully achieves a balance between computational efficiency, inference performance, and resource utilization, making it a viable solution for low-power AI applications in IoT, autonomous systems, and edge computing. However, we acknowledge that this analysis is based on a single hardware platform and a fixed DPU configuration (DPUCZDX8G B1600), which limits the generalizability of the results. The resource usage figures serve primarily as a reference point for evaluating the feasibility of real-world deployments on similar platforms.

3.4 Comparison of Results with Recent Works

Table 4 presents an illustrative comparative analysis of our FPGA-based CNN implementations against recent state-of-the-art works. The selected references include studies that implemented CNN models on FPGAs and reported all of the following metrics: inference latency, classification accuracy, and hardware resource utilization to highlight the trade-offs between resource usage and performance, and to demonstrate the competency of our approach in achieving an optimal balance through careful model selection, quantization, and hardware-aware implementation. Although the references involve different CNN architectures, datasets

Table 4 Performance summary of the proposed design and comparison with the state-of-the-art

		[8]	[15]	[23]	[20]	This Work
Platform		Zynq XC7Z020	Xilinx Zynq UltraScale+MPSoC ZCU104	Zynq XC7Z020	Zynq UltraScale+MPSoC ZU3EG A484	Zynq UltraScale+MPSoC ZU3EG A484
Frequency (MHz)		214	300	667	325	325
DL model		SqeezeNet	CDRNet	Custom 84-layer ResNet	VGG16 / VGG19	*MbN / **SqN / ***ResN18
Precision (bit)		8	8	8	8	8
Dataset		ImageNet	CIFAR-10	CIFAR-10	CIFAR-10	CIFAR-10
Resolution		224×224	32×32	32×32	32×32	224×224
Implementation method		HDL	Vitis AI	Vitis AI	Vitis AI	Vitis AI
Inference latency (ms/frame)		364	1.712	5.2	0.652 / 0.846	10.04 / 6.28 / 19.95
Baseline		NVIDIA TX1	Intel i5-6600 / NVIDIA GeForce RTX 3090	NVIDIA GeForce RTX 2070 SUPER (Turing)	Intel i5-13400 F 2.50 GHz	CPU / GPU
Speed up		6×	3.33× / 2.61×	0.06×	7.3× / 6.61×	*3.31× / **2.65× / ***2.02× / *0.13× / **0.16× / ***0.06×
Top-1 Accuracy (%)		57.27	58.76	92.5[1]	89.54 / 87.46	91.05 / 89.27 / 94.73
Top-5 Accuracy (%)		80.35	N/A	N/A	99.54 / 98.97	99.86 / 99.71 / 99.93
Resource utilization	BRAM	85.5 (61%)	210 (67.31%)	127.5 (91.1%)	126 (58.3%)	126 (58.3%)
	LUT	29,867 (56%)	105,845 (45.94%)	40,712 (76.5%)	38,418 (54.5%)	38,418 (54.5%)
	DSP	190 (86.4%)	1420 (82.18%)	311 (141.0%)	326 (90.6%)	326 (90.6%)
	FF	35,489 (33%)	198,725 (43.13%)	82,730 (77.8%)	58,831 (41.7%)	58,831 (41.7%)

[1] Recognition precision

(primarily CIFAR-10 or ImageNet), and platforms, they were chosen to provide a broad view of the performance-efficiency trade-offs achievable in FPGA-based AI acceleration. We provide this comparison to illustrate general trends in balancing accuracy, latency, and resource utilization. Based on the reported data, our approach shows clear advantages in achieving high classification accuracy while maintaining efficient inference speed and optimized hardware utilization—even when processing significantly larger image resolutions.

Our SqueezeNet implementation outperforms the work in [8], which also deployed the same DL model on an FPGA. While [8] reported a Top-1 accuracy of only 57.27% and a Top-5 accuracy of 80.35%, our implementation achieves 89.27% and 99.71%, respectively, reflecting a remarkable improvement in classification performance. Furthermore, while processing images with the same resolution used in [8], our implementation achieves an inference latency of 6.28 ms, whereas [8] reported a significantly slower 364 ms per frame, demonstrating our model's superior optimization and efficiency.

Also, our ResNet18 implementation surpasses the work in [23], which employed a custom 84-layer ResNet model trained on CIFAR-10. While [23] achieved 92.5% recognition precision, our ResNet18 reaches 94.73%, marking a substantial improvement in recognition performance. Additionally, our inference latency of 19.95 ms per frame, despite processing a 224×224 image resolution compared to 32×32 in [23], underscores the scalability and efficiency of our implementation. It should be highlighted that the mentioned superiority of our work has been achieved with a significant improvement in resource utilization in comparison with [23].

Compared to our previous work [20], our current implementations demonstrate significant improvements, benefiting from the transition to the PyTorch framework within Vitis AI instead of TensorFlow 2. In [20], our VGG16 and VGG19 implementations achieved Top-1 accuracies of 89.54% and 87.46%, respectively. However, by leveraging PyTorch, we achieve a more optimized and flexible model deployment, leading to higher classification accuracy in our ResNet18, SqueezeNet, and MobileNetV2 models, along with better inference speed and more efficient hardware utilization.

Moreover, compared to the work in [15], which implemented the CDRNet model [28] on an MPSoC ZCU104 platform using PyTorch framework, our approach demonstrates significant advantages in both accuracy and efficiency. While [15] achieved a Top-1 accuracy of 58.76% on CIFAR-10, our ResNet18, SqueezeNet, and MobileNetV2 implementations surpass this with Top-1 accuracies of 94.73%, 89.27%, and 91.05%, respectively. Additionally, although [15] reported a relatively low inference latency of 1.712 ms per frame, this was achieved for a small 32×32 image resolution. In contrast, our models process significantly larger 224×224 images while maintaining competitive inference times, demonstrating the efficiency of our implementation under increased input complexity. This suggests that our approach is capable of handling more demanding workloads without incurring substantial latency penalties. Furthermore, our hardware resource utilization remains well-optimized despite the increased computational demands, making our design more suitable for real-world applications that require high accuracy and support for larger input sizes.

We note that the works presented in Table 4 involve different CNN models and deployment targets, which can influence performance metrics independently of implementation technology. Therefore, these comparisons should be interpreted as indicative of broader trends rather than as a strict one-to-one benchmark. Additionally, one of the key challenges in comparing FPGA-based DL implementations is the lack of standardized evaluation across different studies. Many prior works either optimize for inference speed at the expense of accuracy or focus on minimizing resource usage without considering computational efficiency. Moreover, crucial details such as dataset resolution, hardware configurations, and training steps are often omitted, making direct comparisons difficult and limiting reproducibility. Despite these inconsistencies, our work establishes a new benchmark by achieving an optimal balance between classification accuracy, inference latency, and hardware efficiency while processing significantly larger image sizes than most related works. These results make our design highly competitive for real-world AI applications requiring high-performance, low-power FPGA-based DL solutions.

4 Discussion

The acceleration of DL models on FPGAs offers a powerful solution for real-world applications that demand high computational efficiency, low power consumption, and real-time processing. Unlike CPUs and GPUs, FPGAs enable hardware-aware model quantization and optimization, allowing models to achieve superior performance while minimizing resource utilization. These optimizations are particularly advantageous for edge AI applications such as autonomous vehicles and IoT, where energy efficiency and low-latency inference are critical. In this work, we have carefully balanced the trade-offs between accuracy, inference speed, and resource utilization by leveraging model quantization and FPGA-specific optimizations within Vitis AI, ensuring an efficient and scalable deployment without compromising

classification performance. Our implementation demonstrates that FPGA-based DL can bridge the gap between high-speed GPU inference and energy-efficient embedded AI by significantly reducing inference latency compared to CPUs while maintaining a well-optimized resource footprint.

This work demonstrates the effective deployment of lightweight yet high-performance CNN models on FPGAs, achieving a strong balance between accuracy, inference speed, and resource utilization. The implementations of SqueezeNet, MobileNetV2, and ResNet18 outperform prior works in both accuracy and efficiency, with the added advantage of processing significantly larger image sizes without substantial latency penalties. The transition to PyTorch within Vitis AI enhances flexibility, allowing for more optimized model deployment and broader applicability in comparison with Tensorflow2 framework.

The results highlight the inherent trade-off between inference latency and classification accuracy among the implemented models. ResNet18, with the highest accuracy, requires more runtime, making it ideal for applications where accuracy is paramount. In contrast, SqueezeNet, the most lightweight model, achieves the fastest inference, though at a slightly lower accuracy. MobileNetV2 offers a balanced solution, maintaining competitive accuracy with a moderate inference time, making it ideal for applications demanding both efficiency and performance. These findings reinforce the versatility of our FPGA-based DL accelerator, demonstrating its adaptability for various computer vision tasks where real-time decision-making and high accuracy must be achieved simultaneously.

Beyond image classification, our work has broader implications for AI-driven applications. Future work could explore deploying more advanced architectures such as vision transformers on FPGAs to further improve accuracy and efficiency for more sophisticated applications. Additionally, integrating CNNs as backbones for models like UNET and YOLO would enable real-time semantic segmentation and object detection, broadening FPGA-based AI applications. Scaling these implementations for high-resolution datasets requiring real-time classification such as Cityscapes [29] and KITTI [30] will further validate their generalizability for real-world scenarios.

5 Conclusion

Deploying DL models on resource-constrained hardware presents challenges due to high computational demands, memory limitations, and power constraints. FPGAs provide a compelling solution, balancing accuracy, latency, and resource efficiency through hardware-aware optimization and quantization. This work successfully implements and optimizes MobileNetV2, SqueezeNet, and ResNet18 on FPGA using the PyTorch framework within Vitis AI, demonstrating a scalable and efficient DL acceleration approach. Our models achieve Top-1 accuracies of 91.05% (MobileNetV2), 89.27% (SqueezeNet), and 94.73% (ResNet18), while maintaining low inference latencies of 10.04 ms, 6.28 ms, and 19.95 ms per frame, respectively. Our approach efficiently processes 224×224 images without significant accuracy degradation, reinforcing its suitability for real-world applications. Compared to CPU implementations, our FPGA models achieve substantial speedups while offering a power-efficient alternative to GPUs for mobile applications. Future work could explore advanced architectures like vision transformers, developing real-time computer vision applications such as object detection and semantic segmentation based on CNN models, and employing high-resolution datasets, further extending the impact of FPGA-based DL acceleration in edge computing applications.

Author Contributions Ahmad Mouri Zadeh Khaki contributed to Conceptualization, Methodology, Software, Data curation, Formal analysis, Visualization, and writing–original draft. Ahyoung Choi contributed to Funding acquisition, Investigation, Resources, Supervision, Validation, and Writing–review & editing. All authors have read and agreed to the published version of the manuscript.

Funding This work was supported by the Gachon University research fund of 2024.(GCU-202404610001).

Data Availability Data will be made available on reasonable request.

Declarations

Competing Interest The authors have no relevant financial or non-financial interests to disclose.

References

1. Akkad, G., Mansour, A., & Inaty, E. (2024). Embedded deep learning accelerators: A survey on recent advances. *IEEE Trans Artif Intell*, *5*, 1954–1972. https://doi.org/10.1109/TAAI.2024.3298745
2. Shawahna, A., Sait, S. M., & El-Maleh, A. (2018). FPGA-based accelerators of deep learning networks for learning and classification: A review. *Ieee Access: Practical Innovations, Open Solutions*, *7*, 7823–7859. https://doi.org/10.1109/ACCESS.2018.2882104
3. Saidi, A., Ben Othman, S., Dhouibi, M., & Ben Saoud, S. (2021). FPGA-based implementation of classification techniques: A survey. *Integration*, *81*, 280–299. https://doi.org/10.1016/j.vlsi.2021.07.010
4. Mouri Zadeh Khaki, A., Yang, S., Kim, H., Sing, A., & Lee, B. G. (2023). A resource-efficient multi-function embedded eye tracker system implemented on FPGA. *Ieee Access: Practical Innovations, Open Solutions*, *11*, 142931–142939. https://doi.org/10.1109/ACCESS.2023.3291234

5. Mouri Zadeh Khaki, A., Farshidi, E., Ansari Asl, K., Ali, S. H. M., & Othman, M. (2021). Design and analysis of a multirate 5-bit high-order 52 Fsrms $\Delta\sum$ time-to-digital converter implemented on 40 Nm altera stratix IV FPGA. *Ieee Access: Practical Innovations, Open Solutions*, *9*, 128117–128125. https://doi.org/10.1109/ACCESS.2021.3109876
6. Mouri Zadeh Khaki, A., Farshidi, E., & Ansari Asl, K. (2021). Design and FPGA implementation of a multirate $\Delta\sum$ time-to-digital converter with third-order noise-shaping. *Microelectronics Journal*, *108*, 104982. https://doi.org/10.1016/j.mejo.2021.104982
7. Mouri Zadeh Khaki, A., Farshidi, E., & Ansari Asl, K. (2019). A novel low-power FPGA-based 1–1 MASH $\Delta\sum$ time-to-digital converte employing one counter for both stages. *J Elec Comput Eng Innov*, *7*, 173–182. https://doi.org/10.22061/jecei.2019.5493
8. Guo, K., et al. (2018). Angel-Eye: A complete design flow for mapping CNN onto embedded FPGA. *IEEE Trans Comput-Aided Des Integr Circuits Syst*, *37*, 35–47. https://doi.org/10.1109/TCAD.2017.2765526
9. Messaoud, S., Bouaafia, S., Maraoui, A., Ammari, A. C., Khriji, L., & Machhout, M. (2022). Deep convolutional neural networks-based hardware-software on-chip system for computer vision application. *Computers & Electrical Engineering*, *98*, 107671. https://doi.org/10.1016/j.compeleceng.2022.107671
10. Saglam, S., & Bayar, S. (2024). Hardware design of lightweight binary classification algorithms for small-size images on FPGA. *Ieee Access: Practical Innovations, Open Solutions*, *12*, 57225–57235. https://doi.org/10.1109/ACCESS.2024.3334567
11. Umuroglu, Y. (2017). FINN: A framework for fast, scalable binarized neural network inference. In: Proc 2017 ACM/SIGDA Int Symp Field-Programmable Gate Arrays (FPGA '17), Association for Computing Machinery, New York, NY, USA, pp 65–74. https://doi.org/10.1145/3020078.3021744
12. Syed, R. T., Andjelkovic, M., Ulbricht, M., & Krstic, M. (2023). Towards reconfigurable CNN accelerator for FPGA implementation. *Ieee Transactions on Circuits and Systems Ii: Express Briefs*, *70*, 1249–1253. https://doi.org/10.1109/TCSII.2023.3298124
13. Wang, X., et al. (2023). FD-CNN: A frequency-domain FPGA acceleration scheme for CNN-based image-processing applications. *ACM Trans Embed Comput Syst*, *22*, 1–30. https://doi.org/10.1145/3569001
14. Vitis, A. I., & User Guide (UG1414). (2024). Retrieved November 17, from https://docs.amd.com/r/2.5-English/ug1414-vitis-ai/Vitis-AI-Overview
15. Li, Z., Hong, F. Z., & Yue, C. P. (2024). FPGA-based acceleration of neural network for image classification using vitis AI. arXiv preprint arXiv:2412.20974 [cs.CV].
16. Kljucaric, L., & George, A. D. (2023). Deep learning inferencing with high-performance hardware accelerators. *ACM Trans Intell Syst Technol*, *14*, 1–25. https://doi.org/10.1145/3569002
17. Cilardo, A., Maisto, V., Mazzocca, N., & Di Torrepadula, F. R. (2024). An approach to the systematic characterization of multitask accelerated CNN inference in edge MPSoCs. *ACM Trans Embed Comput Syst*, *23*, 1–25. https://doi.org/10.1145/3572234
18. Baitemirova, M. (2022). Evaluating image classification models for FPGA board status detection [dataset]. https://api.semanticscholar.org/CorpusID:250406541. Accessed 11 Mar 2025.
19. Krizhevsky, A. (2009). Learning multiple layers of features from tiny images. Master's thesis, University of Toronto.
20. Mouri Zadeh Khaki, A., & Choi, A. (2025). Optimizing deep learning acceleration on FPGA for real-time and resource-efficient image classification. *Appl Sci*, *15*, 422. https://doi.org/10.3390/app15010422
21. Kluyver, T. (2016). Jupyter Notebooks– a publishing format for reproducible computational workflows. In F. Loizides, & B. Schmidt (Eds.), *Positioning and power in academic publishing: Players, agents and agendas* (pp. 87–90). IOS.
22. Zhao, R. (2017). Accelerating binarized convolutional neural networks with software-programmable FPGAs. In: Proc 2017 ACM/SIGDA Int Symp Field-Programmable Gate Arrays (FPGA '17), Association for Computing Machinery, New York, NY, USA, pp 15–24. https://doi.org/10.1145/3020078.3021746
23. Ushiroyama, A., Watanabe, M., Watanabe, N., & Nagoya, A. (2022). Convolutional neural network implementations using Vitis AI. In: Proc 2022 IEEE 12th Annu Comput Commun Workshop Conf (CCWC), Las Vegas, NV, USA, pp 1–6. https://doi.org/10.1109/CCWC2022.1234567
24. Liu, Z., et al. (2017). Throughput-optimized FPGA accelerator for deep convolutional neural networks. *ACM Trans Reconfigurable Technol Syst*, *10*, 1–23. https://doi.org/10.1145/3120895
25. Wan, Y., Xie, X., Yi, L., Jiang, B., Chen, J., & Jiang, Y. (2024). Pflow: An end-to-end heterogeneous acceleration framework for CNN inference on FPGAs. *J of Systems Architecture*, *150*, 103113. https://doi.org/10.1016/j.sysarc.2024.103113
26. Hosseinabady, M., Zainol, B., & Nunez-Yanez, M. A. (2019). J. Heterogeneous FPGA+GPU embedded systems: Challenges and opportunities. arXiv preprint arXiv:1901.06331 [cs.DC].
27. Biookaghazadeh, S., Zhao, M., & Ren, F. (2018). Are FPGAs Suitable for Edge Computing? arXiv preprint arXiv:1804.06404 [cs.DC].
28. Hong, Z., & Yue, C. P. (2023). Cross-dimensional refined learning for real-time 3D visual perception from monocular video. In: Proc 2023 IEEE/CVF Int Conf Comput Vis Workshops (ICCVW), Paris, France, pp 2161–2170. https://doi.org/10.1109/ICCVW2023.4567891
29. Cordts, M. (2016). The Cityscapes dataset for semantic urban scene understanding. In: Proc IEEE Conf Comput Vis Pattern Recognit (CVPR), Las Vegas, NV, USA, pp 3213–3223. https://doi.org/10.1109/CVPR.2016.350. Cordts M The Cityscapes data set for semantic urban scene understanding. In: Proc IEEE Conf Comput Vis Pattern Recognit (CVPR), Las Vegas, NV, USA; 2016, pp. 3213-23. https://doi.org/10.1109/CVPR.2016.350.
30. Geiger, A., Lenz, P., & Urtasun, R. (2012). Are we ready for autonomous driving? The KITTI vision benchmark suite. In: Proc IEEE Conf Comput Vis Pattern Recognit (CVPR), Providence, RI, USA, pp 3354–3361. https://doi.org/10.1109/CVPR.2012.6248074

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.